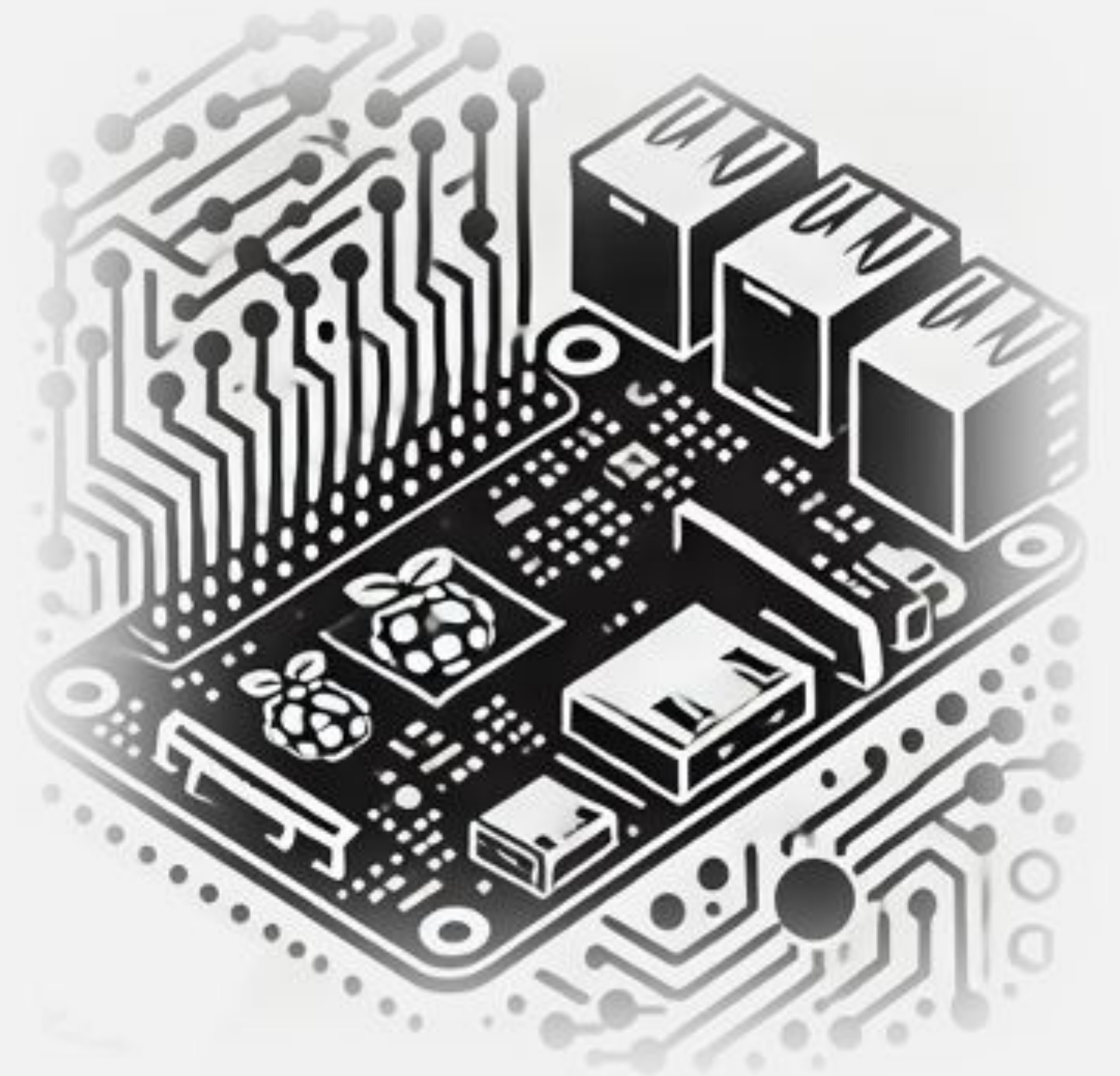


IEST105 – Edge AI

Machine Learning System Engineering

14. Embedded Linux - Implementation



Universidade Federal de Itajubá - UNIFEI
IESTI05 - Machine Learning Systems Engineering

Implementação de um Sistema Linux Embarcado para EdgeAI

Apresentador: Guilherme Fernandes de Oliveira



Objetivos



- Instalar e configurar o sistema operacional Linux Raspberry Pi OS Lite (64-bit) para Raspberry Pi Zero 2W
- Instalar e configurar as dependências para o executar o modelo YOLOv8
- Realizar a integração dos periféricos, câmera OV5647, relógio de tempo real DS1302, sensor de temperatura e umidade SHT20, botão e led.
- Implementar o código para captura, inferência, leitura dos dados e armazenamento em um banco de dados local.

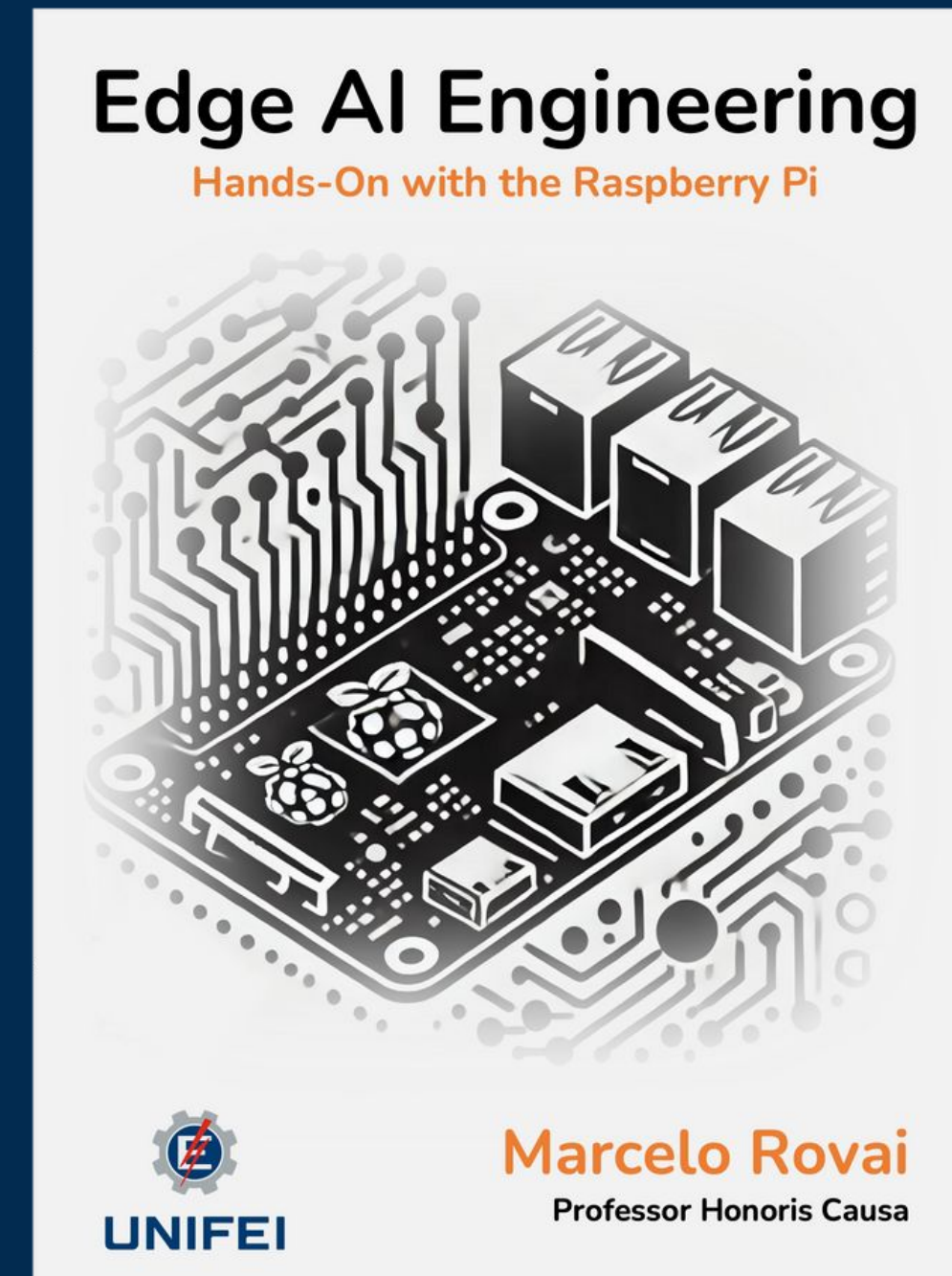
Objetivos



- Otimizar desempenho e consumo de energia no código e nas configurações do sistema.
- Implementar técnicas de sistemas embarcados, como watchdog timer, falha esperada e a recuperação automática do sistema.
- Validar o sistema em campo, utilizando como caso de teste a contagem de abelhas

Metodologia

- Abordagem exploratória
- Documentações
- Sistemas Operacionais Embarcados
- Sistemas Embarcados
- Livro Edge AI Engineering [4]



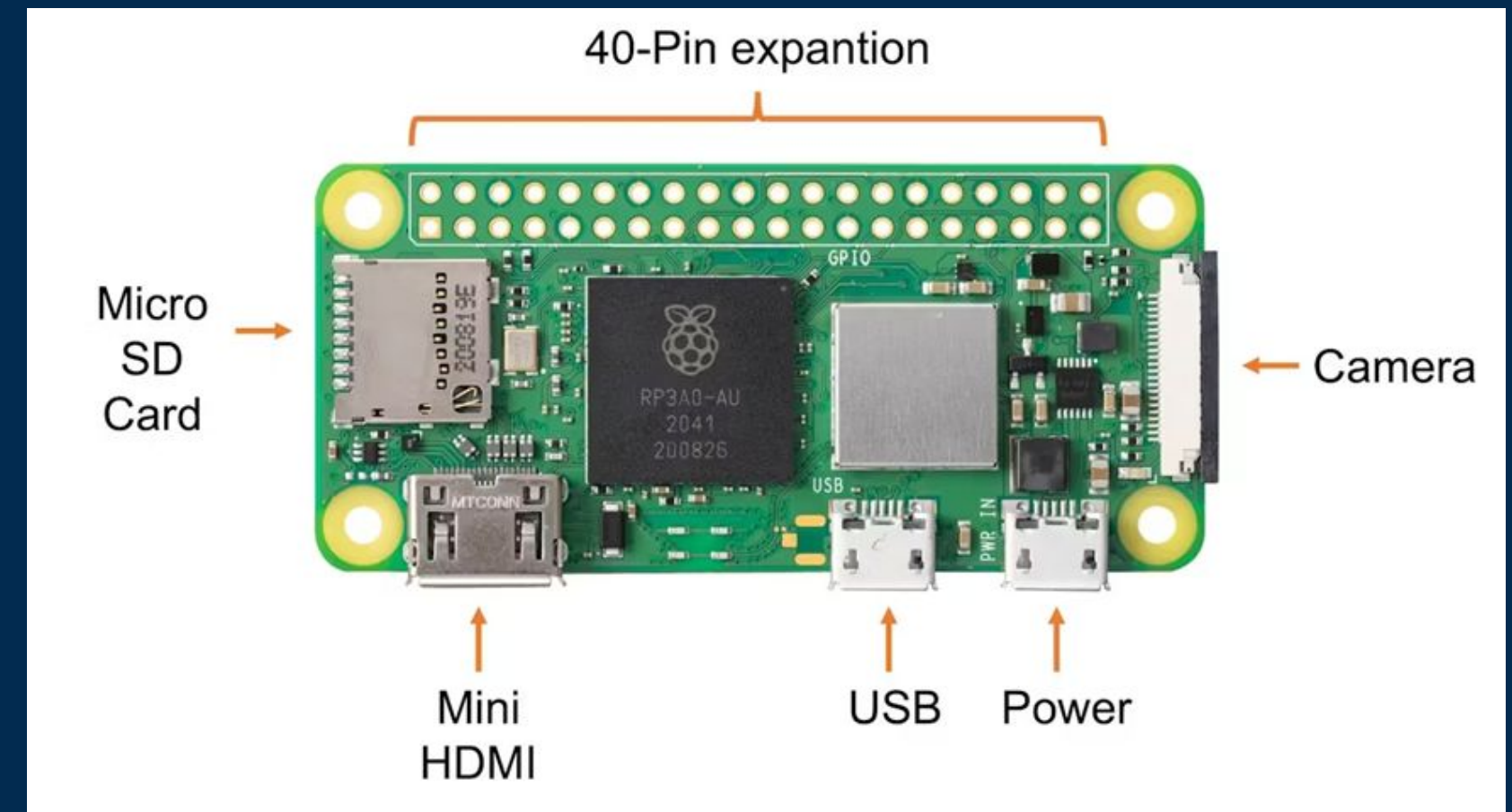
Fonte: Rovai, Marcelo

(2025)[4]

Raspberry Pi Zero 2W



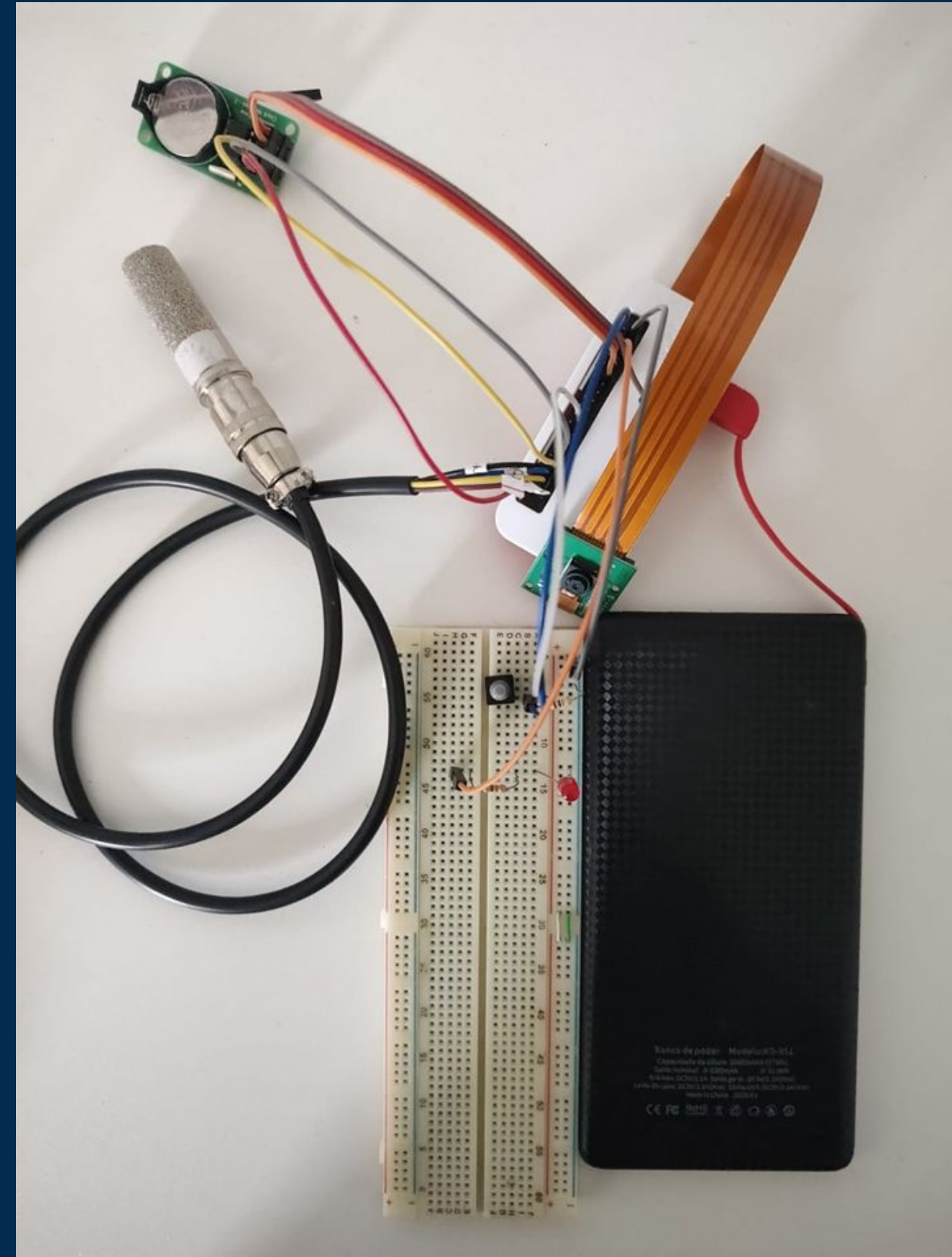
- Processor: 1GHz quad-core 64-bit Arm Cortex-A53 CPU
- RAM: 512MB SDRAM
- Wireless: 2.4GHz 802.11 b/g/n wireless LAN, Bluetooth 4.2, BLE
- Ports: Mini HDMI, micro USB OTG, CSI-2 cameraconnector
- Power: 5V via micro USB port



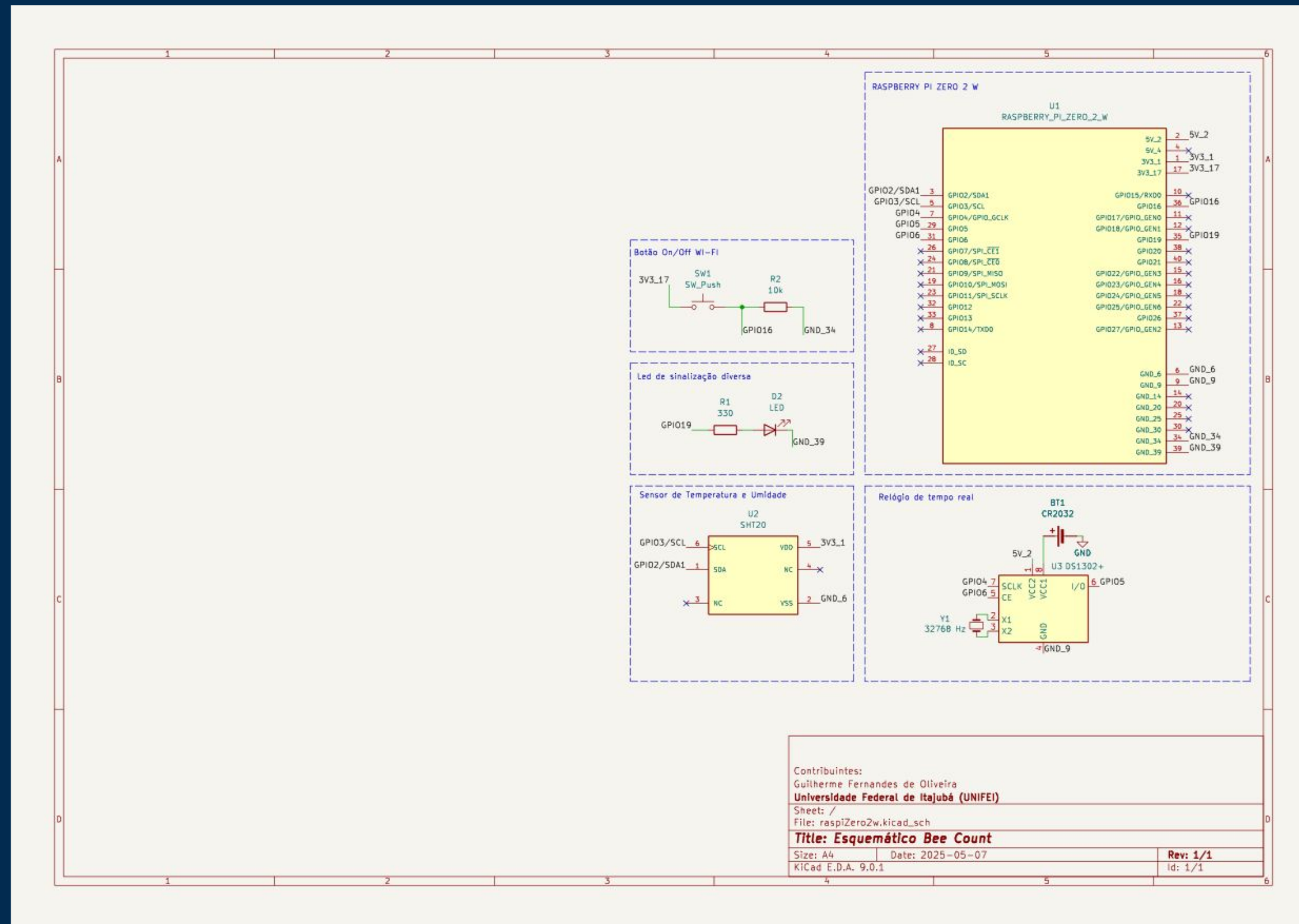
Fonte: Rovai, Marcelo
(2025)[4]

Metodologia

- Raspberry Pi OS Lite (64-bit)
- Câmera OV5647
- Sensor SHT20 [5]
- RTC DS1302 [6]
- Botão
- Led



Metodologia



Metodologia



Metodologia



Aumento da
memória
SWAP



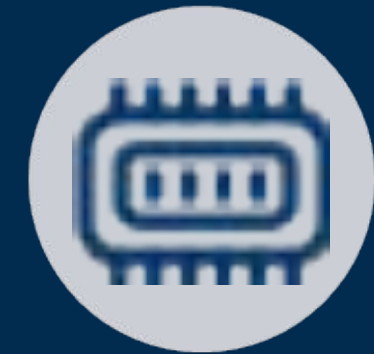
Exportar modelo
para NCNN



Desligamento hdmi,
usb, led e wifi



Salvamento
em lote



Salvamento
temporario na RAM



Pré
carregar
modelo



Threads



Watchdog e
Service

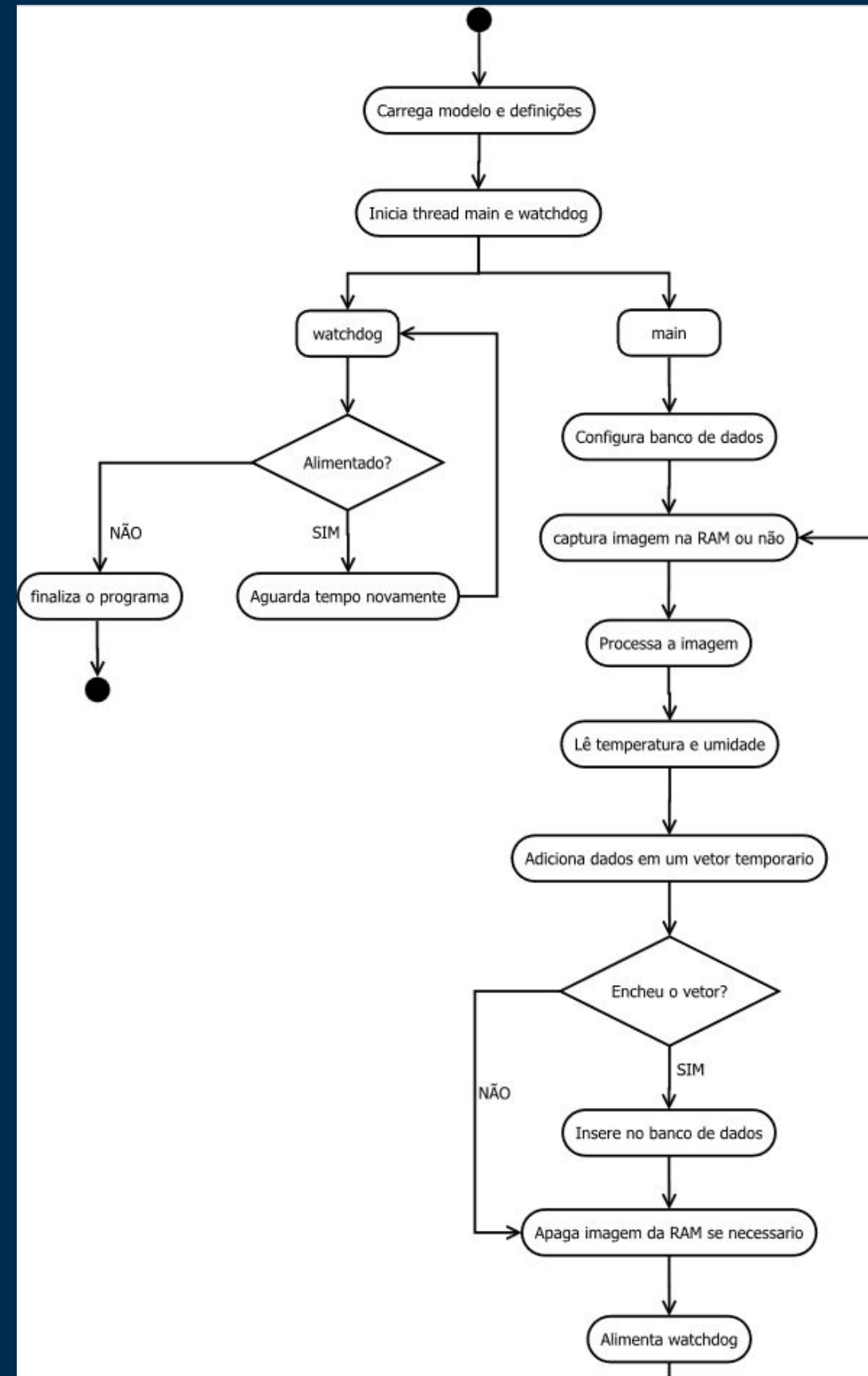


Executemany no
SQLite3



Interrupção
para wifi

Metodologia



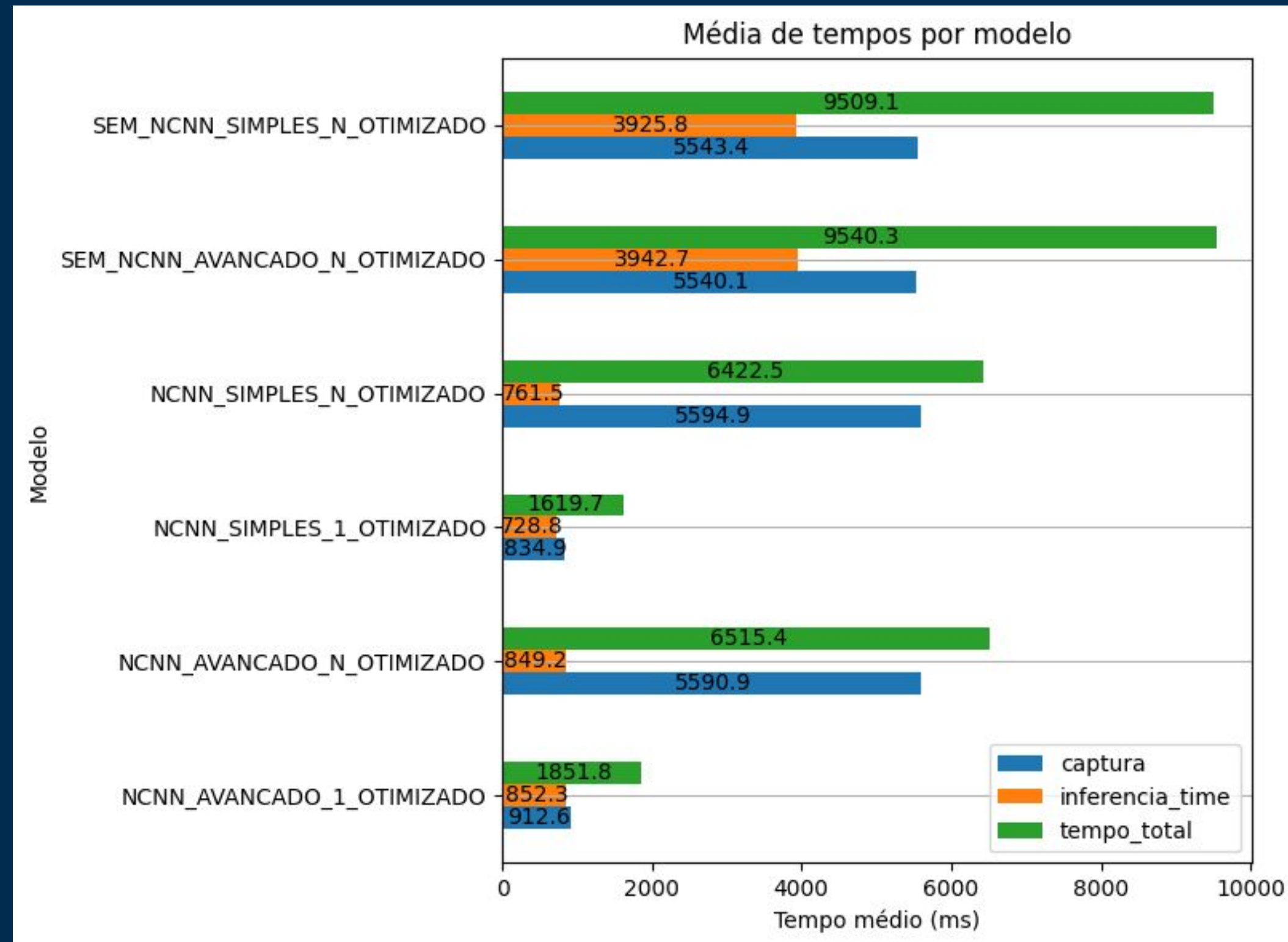
Resultados parciais



```
90|2025-05-26_13-12-27|162|3569|22.41°C|59.69%
91|2025-05-26_13-12-44|162|1839|22.41°C|59.34%
92|2025-05-26_13-12-51|162|850|22.41°C|59.28%
93|2025-05-26_13-12-59|162|860|22.41°C|59.34%
94|2025-05-26_13-13-06|162|856|22.41°C|59.28%
95|2025-05-26_13-13-13|162|849|22.41°C|59.16%
96|2025-05-26_13-13-20|162|839|22.51°C|59.22%
97|2025-05-26_13-13-27|162|834|22.51°C|59.22%
98|2025-05-26_13-13-34|162|851|22.51°C|59.28%
99|2025-05-26_13-13-41|162|839|22.51°C|59.35%
```

```
Loading ../bee_best_ncnn_model for NCNN inference...
Watchdog: processamento travou. Reiniciando processo.
Limpendo GPIO...
(bee) gui@raspiZero:~/Documents/Projeto/YOLO $ █
```

Resultados parciais



Resultados parciais



Resultados parciais



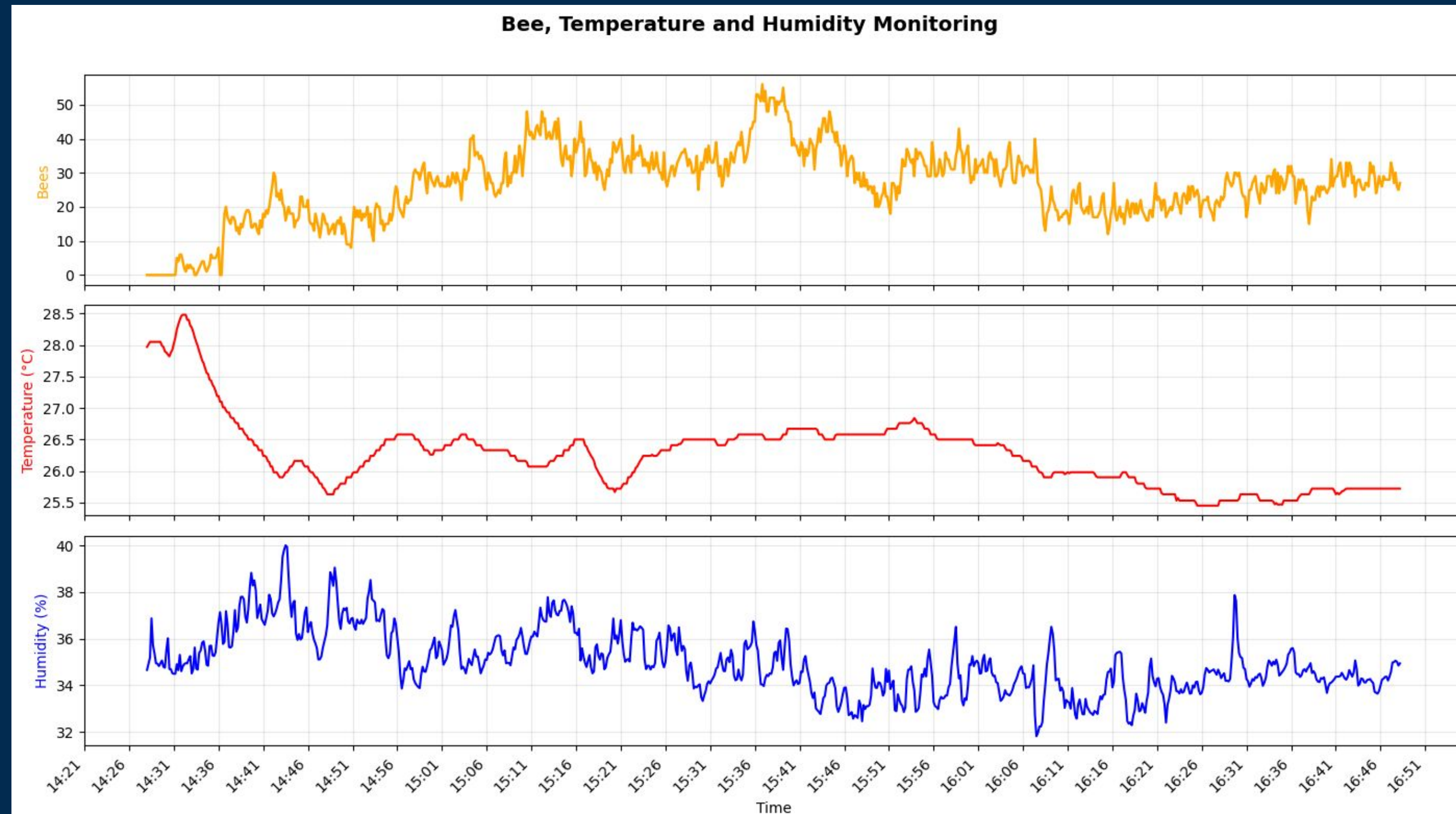
Resultados parciais



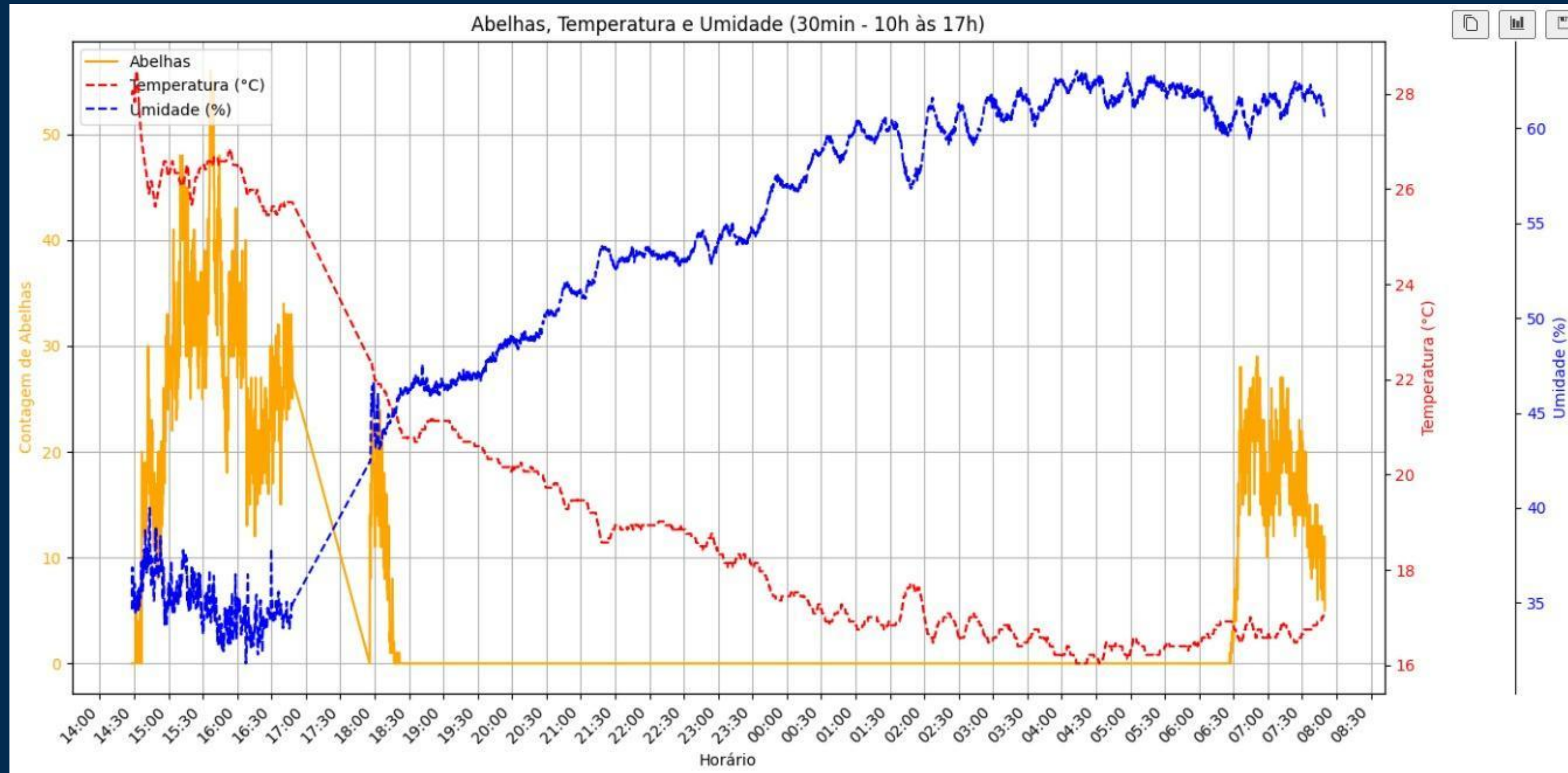
Resultados parciais



Resultados parciais



Resultados parciais



Por dentro do Linux



```
echo "Criando ambiente virtual em ~/bee..."
python3 -m venv ~/bee
source ~/bee/bin/activate
echo "✅ Ambiente virtual criado."

echo "Criando estrutura de pastas..."
mkdir -p ~/Documents/Projeto
cd ~/Documents/Projeto

echo "Copie manualmente a pasta Backup para dentro de ~/Documents/Projeto"
echo "Depois pressione ENTER para continuar..."
read
cd ~/Documents/Projeto/Backup
echo "✅ Estrutura de pastas criada."

echo "Instalando dependências do projeto..."
# pip freeze > requirements.txt
pip install -r requirements.txt
echo "✅ Dependências instaladas."
```

```
echo "Configurando I2C"
sudo apt-get install -y python3-smbus
sudo apt-get install -y i2c-tools
echo "Com a interface raspi-config, vá em \"Interfacing Options\" e ative o I2C."
read -p "Pressione [ENTER] após terminar a edição manual..."
sudo raspi-config
echo "✅ I2C configurado."
```

```
echo "Configurando serviço de inicialização..."
read -p "Pressione [ENTER] após terminar a edição manual..."
sudo nano /etc/systemd/system/bee.service
echo "✅ Edição concluída. Continuando o script..."

echo "Habilitando e iniciando o serviço..."
sudo systemctl daemon-reexec
sudo systemctl daemon-reload
sudo systemctl enable bee.service
sudo systemctl start bee.service
```

Por dentro do Linux



[Unit]

Description=Script Bee - controle de Wi-Fi e sensores

After=multi-user.target # o serviço só inicia *depois* que o multi-user.target foi atingido

StartLimitInterval=50 # monitora 50s para ver quantas vezes falhou

StartLimitBurst=3 # número máximo de falhas dentro da janela acima

[Service]

ExecStart=/home/gui/bee/bin/python /home/gui/Documents/Projeto/Backup/bee_wat.py

Restart=on-failure

RestartSec=5

User=gui

WorkingDirectory=/home/gui/Documents/Projeto/Backup

StandardOutput=journal

StandardError=journal

Environment=PYTHONUNBUFFERED=1 #prints aparecem imediatamente no log

#WatchdogSec=50s

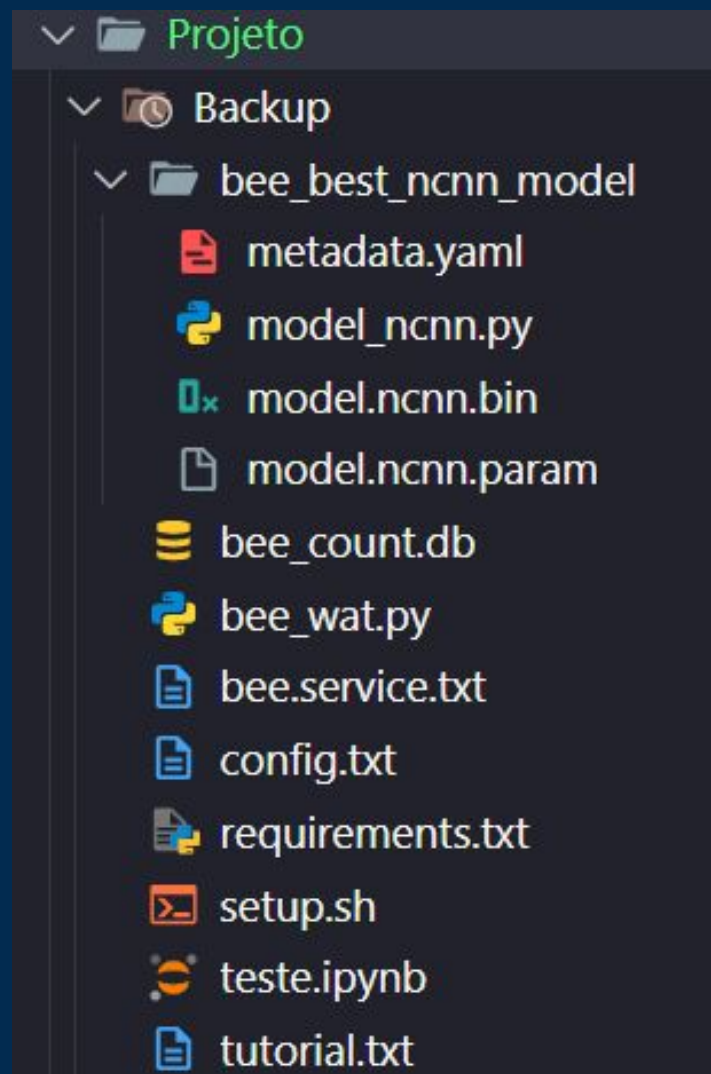
20

[Install]

WantedBy=multi-user.target # faz o serviço iniciar junto com o multi-user.target

```
# Configurar servico
which python
/home/gui/gpio/bin/python
pwd
/home/gui/Documents/Projeto/YOLO/teste.py
sudo nano /etc/systemd/system/bee.service
```

Por dentro do Linux



```
# === BIBLIOTECAS ===
from ultralytics import YOLO
from ds1302 import DS1302
from sht20 import SHT20
import sqlite3
import subprocess
import os
import time
import threading
import RPi.GPIO as GPIO

# === CONFIGURAÇÕES ===
MODEL_PATH = './bee_best_ncnn_model'
CAPTURE_DIR = 'capture/'
PROCESSED_DIR = "processed_images/"
DB_PATH = 'bee_count.db'
BATCH_SIZE = 5
CAPTURE_RAM = False
SAVE_IMAGE = True
WATCHDOG_TIMEOUT = 50 # segundos

BOTA0 = 16 # GPIO16 -> Pino 36
LED = 19 # GPIO19 -> Pino 35

# === Configuracao GPIO para BOTA0 e LED ===
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(BOTA0, GPIO.IN, pull_up_down=GPIO.PUD_DOWN)
GPIO.setup(LED, GPIO.OUT)

event = threading.Event()

# Criar diretórios se não existir
os.makedirs(CAPTURE_DIR, exist_ok=True)
os.makedirs(PROCESSED_DIR, exist_ok=True)

# Definição dos pinos GPIO
CLK_PIN = 4 # GPIO4 -> Pino 7
DAT_PIN = 5 # GPIO5 -> Pino 29
RST_PIN = 6 # GPIO6 -> Pino 31

# === INICIALIZACAO ===
rtc = DS1302(CLK_PIN, DAT_PIN, RST_PIN)
sht20 = SHT20(port=1, resolution=SHT20.TEMP_RES_11bit)

# Carregar modelo YOLO
model = YOLO(MODEL_PATH, task="detect")
```

```
# === LOOP PRINCIPAL ===
def main():
    try:
        setup_database()
        batch = []
        while True:
            print("Começando...")
            t_start = time.perf_counter()
            # 1. Captura
            image_path = capture(ram=CAPTURE_RAM)
            # 2. Processa a imagem
            inference_time, num_bees, timestamp = process_image(image_path, save=SAVE_IMAGE)
            print(f"Inference time {inference_time}")
            print(f"Bee count: {num_bees}")
            # 3. Lê temperatura e umidade
            temp, humid = getTemperatureHumidity()
            print(f"Temperature: {temp:.2f}°C")
            print(f"Humidity: {humid:.2f}%")
            # 4. Adiciona os dados ao batch
            batch.append((timestamp, num_bees, inference_time, f"{temp:.2f}°C", f"{humid:.2f}%"))
            if len(batch) >= BATCH_SIZE:
                insert_batch(batch)
                batch.clear()
            # 5. Apaga a imagem da RAM se necessário
            if CAPTURE_RAM:
                clearRamImage(image_path)
            # 6. Alimenta o watchdog
            event.set()
            # 7. Próxima captura
            print("\n=====")
            t_end = time.perf_counter()
            print(f"Processamento concluído em {round(t_end - t_start, 3)}s")
            pisca_led()

        # Permite interromper o programa com Ctrl+C
    except KeyboardInterrupt:
        print("Programa interrompido pelo usuário.")
        if batch:
            insert_batch(batch)
    except Exception as e:
        print(f"Erro inesperado: {e}")
    finally:
        cleanup()
        print("Limpeza concluída. Encerrando o programa.")
```

Por dentro do Linux



```
✓ if __name__ == "__main__":  
    t1 = threading.Thread(target=watchdog, daemon=True)  
    t2 = threading.Thread(target=main)  
  
    t1.start()  
    t2.start()  
  
    t2.join()
```

```
✓ def watchdog():  
    while True:  
        signaled = event.wait(WATCHDOG_TIMEOUT)  
        if signaled:  
            event.clear()  
        else:  
            print("Watchdog: processamento travou. Reiniciando processo.")  
            cleanup()  
            os._exit(1)
```

```
✓ def process_image(full_path, save=False):  
    '''realiza inferencia'''  
    ✓ result = model.predict(  
        full_path,  
        save=save,  
        imgsz=640,  
        conf=0.3,  
        iou=0.3,  
        show_labels=False,  
        show_boxes=True,  
        verbose=False,  
        project=".",  
        name=PROCESSED_DIR,  
        exist_ok=True  
    )  
    # save the results  
    num_beas = len(result[0].boxes.cls)  
    timestamp = rtc.getDateTime("file")  
    inference_time = int(result[0].speed['inference'])  
    return inference_time, num_beas, timestamp
```

Referências

- [1] YUGANDHARA, R. Y. **Computer Vision Market Trend, Share, Size Growth Report**, 2023. Disponível em: <https://www.researchgate.net/publication/369541205>. Acesso em: 20 mai. 2025.
- [2] PYTECH ACADEMY. Edge AI: The rise of on-device intelligence and faster, more responsive decisions. **Medium**, 2024. Disponível em: <https://pytechacademy.medium.com/edge-ai-the-rise-of-on-device-intelligence-and-faster-more-responsive-decisions-b577b39f84d2>. Acesso em: 16 mai. 2025.
- [3] LIMA, Marcos Venâncio et al. Panorama do aumento da temperatura no planeta e os impactos sobre as abelhas. **Revista Científica de Produção Animal**, [S.l], v. 20, n. 2, p. 71-76, 2019. INSS 2176-4158. DOI: <https://dx.doi.org/10.5935/2176-4158/rcpa.v20n2p71-76>. Disponível em: <https://periodicos.ufpb.br/index.php/rcpa/article/view/45858>. Acesso em: 15 mai. 2025
- [4] ROVAI, Marcelo. **Edge AI Engineering**: Hands-on with the Raspberry Pi, 2025. Disponível em: https://mjrovai.github.io/EdgeML_Made_Ease_ebook/raspi/setup/setup.html. Acesso em: 16 mai. 2025
- [5] FERNANDES, Guilherme. **SHT20-PY**. GitHub, 2025. Disponível em: <https://github.com/guilhermefernandesk/SHT20-PY>. Acesso em: 23 mai. 2025
- [6] FERNANDES, Guilherme. **RTC-DS1302-PY**. GitHub, 2025. Disponível em: <https://github.com/guilhermefernandesk/RTC-DS1302-PY>. Acesso em: 23 mai. 2025

Obrigado pela atenção



Guilherme Fernandes de oliveira
Engenharia Eletrônica



guilhermeferoliv@gmail.com



Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br



UNIFEI